



Full length article

MDVul: A semantic-based complex dependency code vulnerability detection using fusion path[☆]Zhequ Sun^{a,b}, Siyu Wang^{a,b}, Ning Lu^{a,b,c,*}, Wenbo Shi^{a,b}, Zhiqian Liu^d^a College of Computer Science and Engineering, Northeastern University, Shenyang, China^b Hebei Key Laboratory of Marine Perception Network and Data Processing, Northeastern University at Qinhuangdao, 066004, Hebei Province, China^c School of Computer Science and Technology, Xidian University, Xi'an, China^d College of Cyber Security, Jinan University, Guangzhou, China

ARTICLE INFO

Keywords:

Vulnerability detection

Source code vulnerability detection

Complex dependency code

Deep learning

ABSTRACT

On one hand, the reuse of open source code can greatly improve the efficiency of software development; on the other hand, its extended source code functionality would lead to complex dependencies, which also increases the risk of potential vulnerabilities. Although source code vulnerability detection can effectively avoid such risks, existing related methods suffers the issue of detection efficiency, due to the inefficient code representation and insufficient semantic learning capability. For this, we propose MDVul, a vulnerability detection method for complex dependency source code. Specifically, we design a variable-based multiple-dependency flow reconstruction to transform the code into fusion paths. Moreover, we use UniXcoder to embed code representations and further design attention-based bidirectional gated recurrent to improve the ability to learn semantics. To achieve the classification, we use the KAN model. The results show that MDVul outperforms the best baseline method in terms of precision, accuracy, recall, and F1 score by 7.64%, 9.63%, 5.86%, and 7.96%, respectively.

1. Introduction

With the widespread reuse of open source code and the continued expansion of project functionality, the number of code dependencies would increase quickly. Normally, we term the function contained the excessive control and data dependencies as the complex dependency code. According to the recent open-source software code risk report, such complex dependency cannot only make it difficult to maintain, test, and secure the code base, but also increases the risk of potential vulnerabilities [1]. As shown in Fig. 1, this is a typical example of vulnerable complex dependency code (CVE-2024-49854) with a large scale (131 lines) and complex nested conditions (4 layers) [2]. On this basis, for the software developers, it is necessary to timely and accurately detect source code vulnerabilities so as to minimize such risks.

Code vulnerability detection is essentially a classification problem, which intends to construct a detector to identify vulnerability. As shown in Fig. 2, the process includes the training phase and the detection phase. The detector of the training phase directly impacts the accuracy and confidence of the detection phase. Normally, there

are two methods to build vulnerability detectors: similarity-based and semantics-based. The former identifies potential vulnerabilities by comparing similarities between different code structures [3–7]. It is applicable for detecting vulnerabilities in cloned code but fails in complex dependency code. The reasons can be stated: firstly, continuously evolving complex dependency code makes its structure also keep changing, which makes it difficult to realize the structure similarity matching; secondly, complex dependency code often contains complex behaviors and logics within functions (i.e., rich semantics), which increases the difficulty of structural analysis. The latter directly analyzes the code logic and behaviors to detect vulnerabilities, instead of only code structure [8–19]. It is capable of identifying vulnerabilities that are not immediately apparent in similarity-based methods. Therefore, this paper focuses on the semantics-based methods for complex dependency code vulnerability detection.

Semantics-based source code vulnerability detection generally consists of two steps: code representation generation and model learning. There are, however, two challenges in the implementation of MDVul, if we also take into consideration of both the over-complicate code logic and overrich semantic relations of complex dependency code.

[☆] This document is the results of the National Natural Science Foundation of China (62072092, 62072093); the Fundamental Research Funds for the Central Universities, China(N2323023, 2023FGYD001, 2024GFZD003).

* Corresponding author.

E-mail address: luning@neuq.edu.cn (N. Lu).

<https://doi.org/10.1016/j.inffus.2025.103475>

Received 19 March 2025; Received in revised form 12 June 2025; Accepted 27 June 2025

Available online 8 July 2025

1566-2535/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

```

1 static struct bfq_queue *bfq_init_rq(struct request *rq)
2 {
3     . . .
24     if (likely(!new_queue)) {
25         if (bfq_bfqq_coop(bfqq) && bfq_bfqq_split_coop(bfqq) &&
26             !bic->bfq_data[a_idx].stably_merged) {
27             struct bfq_queue *old_bfqq = bfqq;
28             bfqq = bfq_split_bfqq(bic, bfqq);
29             if (!bfqq) {
30                 bfqq = bfq_get_bfqq_handle_split(bfqd, bic, bio,
31                     true, is_sync, NULL);
32                 if (unlikely(bfqq == &bfqd->oom_bfqq))
33                     bfqq_already_existing = true;
34             } else
35                 bfqq_already_existing = true;
36             if (!bfqq_already_existing) {
37                 bfqq->waker_bfqq = old_bfqq->waker_bfqq;
38                 bfqq->tentative_waker_bfqq = NULL;
39                 if (bfqq->waker_bfqq)
40                     hlist_add_head(&bfqq->woken_list_node,
41                         &bfqq->waker_bfqq->woken_list);
42             }
43         }
44     }
45     . . .
131 }

```

Fig. 1. An example of vulnerable complex dependency code.

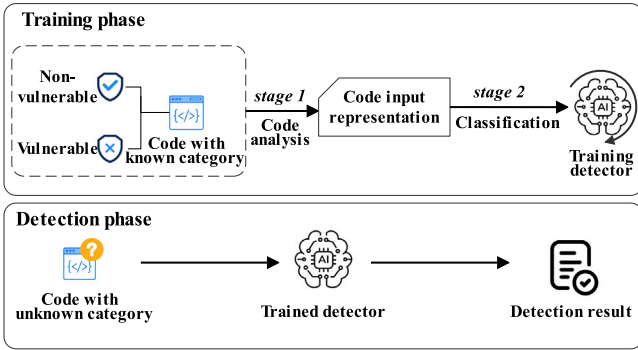


Fig. 2. A brief overview of the vulnerability detection process.

- **Inefficient code representation.** The code representation aims to extract vulnerability-related semantics through preprocessing techniques (primarily code slicing). Code slicing extracts key information from the code and improves vulnerability detection accuracy with limited computing resources [15–17,19]. However, existing methods predominantly employ slicing that terminates at predefined vulnerability syntax patterns (e.g., API calls or sensitive variables). They focus on extracting code semantics related to potential vulnerabilities, but ignore essential semantic relationships and potentially miss vulnerabilities due to incomplete semantics. For complex dependency code, these methods make it difficult to extract complete semantic information from functions with intricate control and data dependencies. Thus, in order to reduce redundancy while preserving essential semantics, MDVul should design an efficient code representation generation.
- **Insufficient feature learning ability.** Model learning extracts underlying information and patterns by analyzing large amounts of data to learn vulnerabilities in semantic relationships. Existing deep learning models typically encode entire code snippets into single vector representations, and then compress semantic information into undifferentiated embedding [12–17,19]. They fused all semantic information into indistinguishable features, which prevented the model from identifying vulnerabilities. When faced with simple code, they may perform well with limited training. However, traditional models struggle to fully understand the semantics and dependencies of complex dependency code when all the semantic information is provided simultaneously, due to lack of the capability to distinguish critical vulnerability-related

semantics from the vast amount of redundant code. Thus, how to design a model that can fully learn to detect complex dependency code vulnerabilities is the second challenge.

To address the above challenges, we propose a vulnerability detection method for complex dependency code, termed as MDVul. Firstly, we design a novel code slicing method that transforms extracted dependency flows into fusion paths. It simplifies dependencies by decomposing function variables into multiple code representations. Secondly, we use the pre-trained model UniXcoder to embed code representations. To learn all the semantic information of the complex dependency code, we leverage the attention-based bidirectional gated recurrent (Attention-BGRU) to aggregate path associations and apply a Kolmogorov-Arnold Networks (KAN) classifier to detect vulnerability. In summary, the main contributions are summarized as the following three-fold.

- We design a variable-centered code slicing method, which reconstructs complex dependency code into multi-dependency fusion paths.
- We design a task-specific model, which encodes each path to extract features, aggregates the semantics of multiple paths and uses KAN for classification.
- We perform comprehensive comparative experiments to evaluate MDVul, and our method improves the precision by 7.64%, accuracy by 9.63%, recall by 5.86%, and F1 score by 7.96%, over the best baseline.

The remainder of this paper is organized as follows. In Section 2, we introduce the related literature. Section 3 defines the detection problem, and Section 4 presents our proposed solution in detail. Section 5 reports the experimental setup and performance analysis. Section 6 offers a discussion, and the final section concludes the paper.

2. Background and related literature

2.1. Complex dependency code vulnerability

Complex dependency code includes the following situations: functions that involve numerous external inputs or internal variables and lack clear control and validation mechanisms, or functions that contain extensive conditional and iterative dependencies. Such code typically involves complex logic, conditionals, and looping dependencies that lead to complex variable manipulation. When facing complex dependencies, developers may not fully understand the internal processing, which can result in overlooked security checks and potential vulnerabilities.

Complex dependency code vulnerabilities are caused by incorrect logical relationships between variables, which can lead to program crashes or malicious exploitation. Due to complex logic and variable interactions, context often becomes unclear. Without full understanding of the code, developers may miss important validation steps or misinterpret critical logic, leading to vulnerabilities. According to the research concepts of CWE, complex dependency code vulnerabilities typically contain the following pillars [20].

- **Improper Control of a Resource Through its Lifetime (CWE-664).** During resource lifecycle management, the system fails to properly control the allocation, usage, and release of resources. Such vulnerabilities often arise when the system or application fails to adequately track, manage, or clean up resources, leading to issues such as Use After Free (CWE-416) and Out-of-bounds Read (CWE-125).
- **Protection Mechanism Failure (CWE-693).** This type of vulnerability refers to the failure or incorrect use of security protection mechanisms, leaving the system unable to effectively defend against attacks. It commonly results in issues such as Cross-site Request Forgery (CWE-352) and Improper Verification of Cryptographic Signature (CWE-347).

- *Improper Neutralization (CWE-707)*. The program fails to properly process or filter internal or external input data. Common neutralization issues include SQL Injection (CWE-89) and Improper Input Validation (CWE-20).

2.2. Related literature

According to the structure and semantics of complex dependency code, existing methods can be divided into two categories: similarity based methods and semantics based methods.

2.2.1. Research on code similarity-based method

Similarity-based methods assume that similar programs may contain the same vulnerabilities. They usually compare known vulnerabilities with tested code to determine whether a program is vulnerable. Code similarity methods are often used in code cloning problems. For example, Jiang used AST to represent the code, embedded it into vectors, and then calculated the similarity between vectors using Euclidean distance to find similar code [3]. Experts discovered that it could also be applied to code vulnerability detection. Li et al. pointed out that no single code similarity algorithm works for all types of vulnerabilities [4]. Therefore, he proposed building the Vulnerability Patch Database (VPD) to store specific differences in vulnerability patches and the Vulnerability Code Instance Database (VCID) to record instances of vulnerability reuse. Then, they used them along with the NVD dataset to construct the CVE-to-algorithm mapping and vulnerability signatures. In the detection phase, given a known vulnerability CVE ID through the CVE-to-algorithm mapping, the detector compared the target code with the chosen code similarity algorithm and the program vulnerability signature to determine whether the target code contains such a vulnerability. VDSimilar used a Siamese network combined with BiLSTM and the attention mechanism to learn pairs of similarity and difference [5]. In the detection phase, the trained model was used to calculate the similarity between the tested and known vulnerability functions. The threshold was used to determine whether there was a vulnerability. Yang et al. proposed to achieve vulnerability detection and patch identification by comparing semantic similarities and differences between vulnerability and patch functions [6]. Han et al. proposed a model that used the cross-attention mechanism inherent in the transformer architecture [7]. This model compared the target source code with the hierarchical structure of the archived vulnerable code to assess the similarity of the target source code.

Similarity-based methods detect potential vulnerabilities by comparing the similarities between the target and known vulnerable code. These methods perform well in code clone detection but may fail if the vulnerable code structure differs significantly from known vulnerabilities. Similarity-based methods highly rely on the quality and quantity of existing vulnerability libraries and patch data.

2.2.2. Research on code semantics-based method

Semantics-based methods can be divided into three methods: symbolic execution, rule-based, and machine learning. Symbolic execution is often combined with constraint solving to detect vulnerabilities. For example, Liang et al. combined symbolic execution with the Z3 SMT solver to detect code vulnerabilities, which can detect divide zero error, pointer overflow, and dead code [9]. Luckow et al. proposed a method of guided symbolic execution to compute bounds on worst-case complexity and generate test values that trigger worst-case behaviors [10]. Tu improves the detection of heap-based vulnerabilities with a boosted symbolic execution engine [11]. While these methods are effective for small-scale functions, they may cause state-space explosion problems in large-scale functions.

Rule-based vulnerability detection methods use vulnerability rules to detect vulnerabilities. These rules were generated by expert manual analysis of various vulnerabilities. Open-source tools such as Flawfinder [21], RATS [22], and ITS4 [23] are rule-based vulnerability detection

tools. While these tools are inexpensive due to simple vulnerability rules, they suffer from a high false positive rate. Chen et al. improved analysis performance by filtering vulnerability rules based on feature objects before checking the source files [24]. The regulations in rule-based vulnerability detection methods depend on human experts, which can result in low accuracy due to subjectivity.

In recent years, deep learning techniques have made significant advances in various fields such as image recognition and natural language processing. They have demonstrated powerful automatic feature learning capabilities. For example, KANs have been proposed to replace MLPs [25], and some researchers have applied them to related fields [26]. As research becomes more complex and data more diverse, deep learning models need to handle multiple types of data. This has driven multimodal learning and heterogeneous graphs to become prominent research topics [27,28]. Following this trend, the application of deep learning in software engineering has gained increasing attention. Because of its ability to automatically learn features, researchers have begun to experiment with combining deep models with code semantics to build vulnerability detectors. They apply the features from many known vulnerabilities to identify potential vulnerabilities in unknown projects [13]. For example, Fu and Tantithamthavorn [12] tokenized the source code using BPE Subword Tokenization [29] and then input it into the pre-trained BERT model. Due to BERT's input limitations, this method poorly detected vulnerabilities in complex dependency code. The Reveal method proposed by Chakraborty et al. converted the code into a code property graph (CPG) as input for the model [13]. It used a GGNN for learning and finally applied MLP for classification detection. Devign converted the code into a joint graph structure of AST, CFG, and DFG [14]. Then, GGNN and convolutional layers were used for feature extraction and classification. However, due to the limitation on the number of nodes in the graph structure, it can only detect short functions and is unsuitable for detecting complex dependency code. In general, vulnerabilities make up a small proportion of the code. Researchers use some rules to find the central point of the code and then extract code fragments to represent the features of the code. The VulDeePecker method proposed by Li et al. used code gadgets to find key points as the "center" of vulnerabilities, then applied BLSTM for feature extraction and classification [15]. They focus on library/API function calls as key points. For each key point, code gadgets were generated by data flow and control flow analysis. Three years later, Li et al. proposed a new detection framework named SySeVR [16]. This method introduced a novel method to extract the center of the program, inspired by regional proposals. It achieved the best training results using BGRU. He decomposed the source code into AST and then analyzed whether the token matched the vulnerability syntax features. Four vulnerability syntax features are extracted here: Library/API Function Call, Array Usage, Pointer Usage, and Arithmetic Expression. Data and control dependencies were extracted for statements that match the vulnerability syntax features. These code fragments were transformed into SeVCs for neural network input. However, this slicing method lost much of the basic code semantics. Zhang et al. used control dependencies between code to decompose the CFG of code fragments into multiple execution paths [18]. Then, he used the pre-trained model CodeBERT [30] and convolutional neural networks [31] to learn path representations. This method could preserve the semantics of the code. However, the method only considered the control dependencies in the code and did not consider the data dependencies. This method led to poor results in detecting some vulnerability types.

Semantic-based vulnerability detection methods identify potential vulnerabilities in the code by analyzing its semantic information. This method is not limited to the structure of the code, and it understands the execution of the program. Compared to similarity-based methods, it can perform better in detecting complex vulnerabilities. Due to the intricacy of complex dependency code, rules-based and symbolic execution-based methods cannot effectively detect vulnerabilities within it. Although deep learning-based code semantic analysis methods have made significant progress in source code vulnerability detection, there is still a lack of effective methods specifically designed to detect vulnerabilities in complex dependency code.

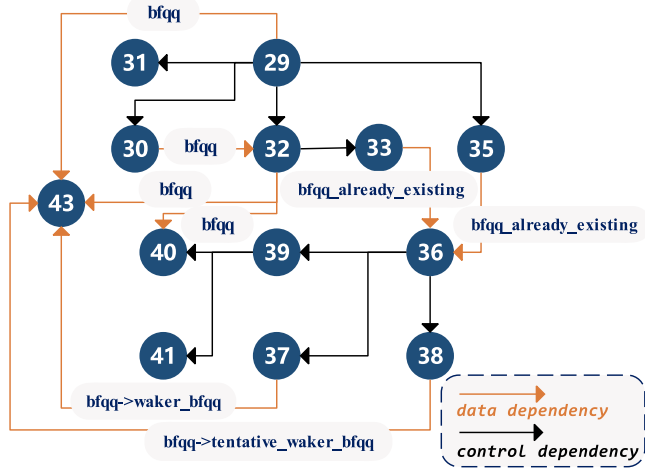


Fig. 3. Code semantics visualization using a graph structure. The graph shows the code semantics from line 29 to line 42 in CVE-2024-49854, where the orange lines represent data dependencies and the black lines represent control dependencies.

3. Problem statement

In this section, we first consider code as the research object. Next, we discuss the definition of code vulnerability detection. Finally, we decompose the problem into two sub-problems.

3.1. Semantic-based source code

Semantics-based source code is a code snippet that contains the results of conditional judgment and data transfer, which are represented through control and data dependencies. The source code function is defined as an ordered set $F = \{c_1, \dots, c_r\}$, where each c_i represents a statement in the function. We define the dependency relationship as $Dep = \{(c_i, c_j) | c_i \rightarrow c_j\}$, which indicates that there is a dependency between c_i and c_j . $r_i = \{c_{i_1}, \dots, c_{i_k}\}$ is a set of dependent paths, where $\forall c_{i_j}, c_{i_{j+1}} \in r_i$, it holds that $(c_{i_j}, c_{i_{j+1}}) \in Dep$. Specifically, r_i^d denotes the data dependency flow between statements, while r_i^c represents the control dependency flow. Additionally, we denote $r_i = r_i^d \cup r_i^c$. We define code semantics as a description of code statements and their dependencies, denoted as Sem . The Sem is formalized as follows:

$$Sem = (F, Dep), \quad Dep = Dep^d \cup Dep^c, \quad (1)$$

where Dep^d describes data dependencies in function F , and Dep^c represents control dependencies.

We visualize the code semantics through the code graph structure. As shown in Fig. 3, it demonstrates the code semantics from line 29 to line 42 in function CVE-2024-49854.

3.2. Problem definition

Code semantic mistakes are the primary cause that distinguishes them from non-vulnerabilities. Source code vulnerability detection is a classification problem based on analyzing the semantics of the code.

Given a set of functions Fun containing multiple function entities F_i , denoted as $Fun = \{F_1, \dots, F_m\}$. Each function entity F_i is associated with a label y_i , where $y_i \in \{y_0, y_1\}$. Code semantics is a set of dependencies between statements constructed through dependency flows, denoted as Sem_i . Correlations between functions and labels are established using known labels and established code semantics. The correlation is a mapping between function entities and their corresponding labels. When an unknown function sample u is given, its code semantics are represented as $\hat{u}$. The established mapping function generates a corresponding predicted classification result $\hat{y}$. Here, $\hat{y}$

Table 1

Main notations.

Notations	Description
F	A function consisting of code statements.
G	The code property graph (CPG) generated from function F using Joern.
Set_v	The set of variables extracted from the graph G .
r_i^d, r_i^c	The data dependency flow and control dependency flow of variable var_i .
r_i	The set of dependency flow statements of var_i , i.e., $r_i^d \cup r_i^c$.
R	The union of all r_i , representing the dependency flow statements of all variables in F .
N	The number of fusion paths.
M	The weight similarity matrix of code statements.
S	A set of code snippets containing N multi-dependency flow fusion paths.

belongs to the class label set $\hat{y} \in \{y_0, y_1\}$. The mapping relationship is formalized as follows:

$$y_i = f(Sem_i) \rightarrow \hat{y} = f(\hat{u}). \quad (2)$$

Definition 1. (Code Semantics Sem_i). Sem_i represents the code semantics contained in function F_i . It can represent the semantic information in the source code.

Definition 2. (Label y). The label is a nominal scale representing the categories of source code, specifically used to indicate the presence of vulnerabilities in the code. Each option represents only one classification. Code can also be assigned to only one of these options. It can be expressed as $y_i \in \{y_0, y_1\}$, where y_0 is non-vulnerable and y_1 is vulnerable.

3.3. Problem decomposition

We decompose the problem into two sub-problems, consisting of code representation generation and model learning.

Sub-problem 1: code representation generation. Considering the complexity of code semantics and the limited computational resources, we need to extract the key information of code semantics. Vulnerability detection involves analyzing the code representation to determine whether it contains potential code vulnerabilities. This process transforms the detection problem into determining the nature of unknown code by analyzing the semantics of code slices with known labels. Semantics-based slicing methods extract key semantic features by analyzing code dependency flow relationships. They reduce irrelevant or redundant information and focus computational resources on the most meaningful parts of the code. Code slicing is a process of removing redundant code semantics and extracting key semantic information from the code.

Code slicing extracts key semantic information from source code by analyzing dependency flows. The original complex dependency code semantics is defined as $Sem = (F, Dep)$. The process of code slicing is formalized as follows:

$$Sem_{slice} = Slice(Sem), \quad (3)$$

where $Sem_{slice} = (S, Dep_{slice})$ represents the code slicing result. It retains the key statements and dependencies after slicing. $S = \{s_1, \dots, s_k\}$ represents the key statements after slicing, where $\forall s_j, s_{j+1} \in S$, it holds that $(s_j, s_{j+1}) \in Dep_{slice}$. $Slice$ is a slicing operation that extracts key semantic information from Sem . The main aim is to reduce redundant code, which can be formalized as $\min(Redundancy(Sem))$, where $Redundancy(Sem)$ represents redundant code.

Sub-problem 2: model learning. To use automated learning technology to learn code semantic information, it is necessary to build a classification model. The detection model aims to establish a functional relationship between the variables F and y . Ultimately, vulnerability

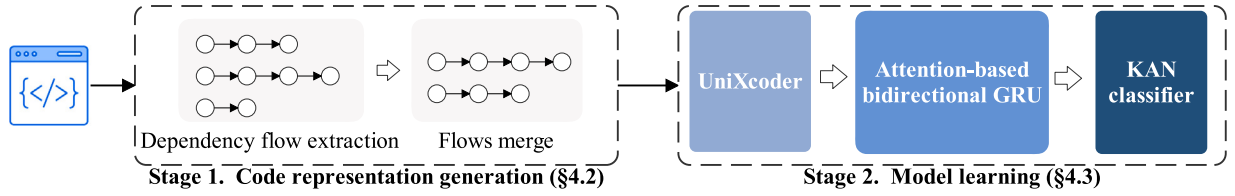


Fig. 4. The framework of MDVul.

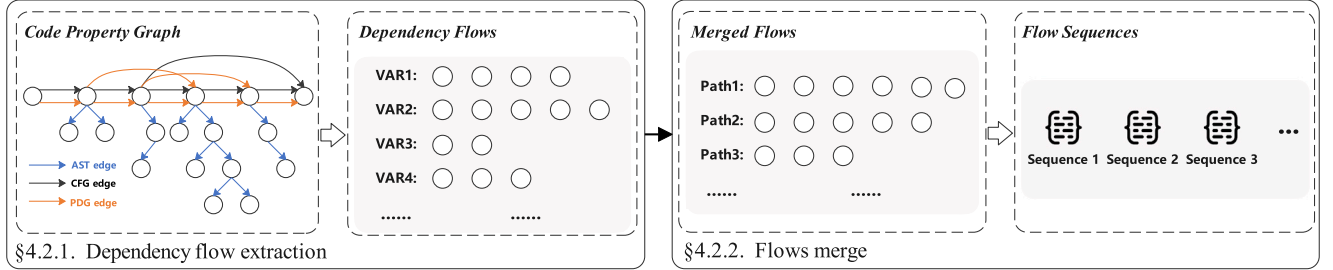


Fig. 5. Multi-dependency flow fusion path reconstruction.

detection aims to achieve binary classification. y is formulated as follows:

$$y = f(\text{Sem}) = g(S; \theta), \quad (4)$$

where g is a classification algorithm. The goal of the classification problem is to find an optimal model parameter θ^* to minimize the loss function $\mathcal{L}(g(S; \theta), y)$. The loss function measures the deviation between the classification result and the true label. The optimization objective is defined as follows:

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(g(S_i; \theta), y_i), \quad (5)$$

where N denotes the number of training samples, S_i denotes the code input representation and y_i represents the true label of F_i .

4. The MDVul

In this section, we introduce the overall framework of the MDVul and how the framework works as a whole by following the main process. Next, we dive into its design details. Table 1 lists the notations used to describe the design of MDVul.

4.1. An overview

By analyzing the causes of vulnerabilities, we proposed the MDVul, with the overall process framework shown in Fig. 4. The framework primarily comprises two key stages: code representation generation and model learning. Firstly, the source code is transformed into a CPG. Then, multiple dependency flow fragments are extracted by leveraging the dependencies among variables in the source code. Subsequently, a greedy algorithm based on code similarity is used to reconstruct these code fragments into multi-dependency flow fusion paths. These paths are input into the model as sequences. Each path sequence is processed by a pre-trained model to encode and extract relevant features. Attention-BGRU is utilized to aggregate the semantic information of the dependent streams. Finally, the KAN is applied to detect vulnerabilities.

4.2. Code representation generation

In this subsection, we design a variable-based code slicing method called a multi-dependency flow fusion path, which aims to decompose code into multiple semantic units. These units are designed to preserve the basic semantics of the code while simplifying its representation,

called multiple dependency flow fusion paths. As shown in Fig. 5, the workflow of the multi-dependency flow fusion path reconstruction is demonstrated.

4.2.1. Dependency flow extraction

Vulnerabilities in programs are caused by inputting specific data into unsafe operations [19]. Due to the inherent flow of variables, dangerous operations may be continuously transmitted through dependency flows. Dangerous operations on variables lead to the propagation of vulnerabilities through their data and control dependencies. We consider variables to be the core of the key semantics of the code. A function often contains several variables. If there is no data or control dependency between two variables, they are redundant within their dependency flows. We need to extract the dependency flow of each variable. Our task at this stage is to preserve the key code semantics and remove redundant code.

Our method focuses on variable-centered code semantics, defined as $\text{Sem}_{\text{key}} = (S, \text{Dep}_{\text{key}})$. Given the original code semantics $\text{Sem} = (F, \text{Dep})$, where $F = \{c_1, \dots, c_r\}$ and $\forall c_k \in F$, we preserve the dependency flows R that are related to variables. R is defined as:

$$R = \bigcup_{i=1}^z r_i, \quad r_i = r_i^d \cup r_i^c, \quad (6)$$

where $\forall c_k \in r_i, c_k \in F$. The core statement set S is defined as the set of statements contained in the dependency flows R . S is formalized as follows:

$$S = \bigcup_{i=1}^z \{c_k \mid c_k \in r_i\}, \quad (7)$$

where $c_k \in F$ for all $c_k \in S$, implying that $S \subseteq F$. Statements in F that are not included in S are defined as redundant statements. They are formalized as follows:

$$F_{\text{Red}} = F \setminus S. \quad (8)$$

Clearly, $|F_{\text{Red}}| \geq 0$. Our method retains the variable-related dependency flow statement set S . It uses S to construct a simplified semantic representation $\text{Sem}_{\text{key}} = (S, \text{Dep}_{\text{key}})$, where Dep_{key} denotes the dependencies within the core statements S . Since S captures the key semantic dependencies of the program, Sem_{key} preserves the integrity of key code semantics while removing semantic redundancy.

Algorithm 1 describes the process of extracting dependency flows. We first generate the CPG for the function F . Then, we extract all the

Algorithm 1: Dependency flow extraction

Input : F : A function consisting of code statements.
Output: R : A set of dependency flow lists.

```

1  $G \leftarrow \text{ConvertToCPG}(F)$ 
2  $\text{Set}_v \leftarrow \text{ExtractVariables}(G)$ 
3 for each variable  $\text{var}_i \in \text{Set}_v$  do
4    $r_i^d \leftarrow \text{DataFlows}(G, \text{var}_i)$ 
5    $r_i^d \leftarrow (r_i^d - r_{i,\text{repeat}}^d)$ 
6    $r_i \leftarrow r_i \cup r_i^d$ 
7   for each statement  $r_s \in r_i^d$  do
8      $S_{cd} \leftarrow \text{StatementsControlledBy}(G, r_s)$ 
9      $S_{cl} \leftarrow \text{ControlStatements}(G, r_s)$ 
10     $r_i^c \leftarrow S_{cd} \cup S_{cl}$ 
11     $r_i \leftarrow r_i \cup r_i^c$ 
12  end
13  if  $r_i$  is not empty then
14     $R[i] \leftarrow r_i$ 
15  end
16 end
17 return  $R$ 

```

variables $\text{Set}_v = \{\text{var}_1, \dots, \text{var}_z\}$ in the graph G to prepare for the multi-dependency flow extraction. After obtaining the data dependency flow r_i^d for the variable var_i , it is added to the set r_i . We then traverse each data dependency statement of the variable var_i to obtain its control dependency statements r_i^c . These statements intersect the sets of statements controlled by and controlling the variable, denoted as S_{cd} and S_{cl} , respectively. We take the union of the data dependency statements r_i^d and the control dependency statements r_i^c for the variable var_i . We then add this union to the set r_i . After iterating over all the variables, we obtain the set of variable-centered dependency flows for the function F , denoted as $R = \{r_1, r_2, \dots, r_z\}$.

4.2.2. Flows merge

The size and count of dependency flows are usually different between code fragments, particularly in complex dependency code. Dependency flows are extracted from code variables and treated as independent entities. In code execution, the data and control dependencies between different variables often have a complex relationship. In other words, dependency flows between variables are interconnected. It indicates that the dependency flows of multiple variables may influence each other or share certain dependency paths. Analyzing code fragments requires considering interactions among dependency flows. We also need to consider the length of each path to avoid excessive computational complexity due to long path lengths. Based on the above considerations, we propose a merging method. Algorithm 2 illustrates the process of merging the extracted dependency flows into multi-dependency flow fusion paths.

Given the set R and the number of reconstructed paths N , we first set initial weights for each dependency flow path, denoted as $W = \{w_1, w_2, \dots, w_z\}$. In particular, the weight decreases as the sequence length increases to balance path lengths. Next, we calculate the similarity between each pair of dependency flows to obtain the weight similarity matrix M , where each element is defined as $M[i][j] = \text{Emergence}(r_i, r_j)$. The emergence weight combines both the structural similarity and weight of the sequences. Specifically, the emergence weight is defined as:

$$\text{Emergence}(r_i, r_j) = \text{Sim}(r_i, r_j) \cdot \text{Weight}(r_i) \cdot \text{Weight}(r_j), \quad (9)$$

where $\text{Weight}(r_i)$ and $\text{Weight}(r_j)$ denote the weights of dependency flows r_i and r_j . $\text{Sim}(r_i, r_j)$ is the similarity between sequences r_i and r_j , defined as:

$$\text{Sim}(r_i, r_j) = \frac{2 \cdot |r_i \cap r_j| + e}{\min(|r_i| + |r_j|) + e}, \quad (10)$$

Algorithm 2: Flows merge

Input : F : A function consisting of code statements;
 R : A set of dependency flow lists;
 N : Number of fusion paths.
Output: S : A set of code snippets with N multi-dependency flow fusion paths.

```

1  $\hat{R} \leftarrow \text{Sort}(R);$ 
2  $n \leftarrow \text{Length}(\hat{R});$ 
3  $W \leftarrow \text{SetDefaultWeights}(\hat{R}, W);$ 
4  $M \leftarrow \text{CalSimilarityMatrix}(\hat{R}, W);$ 
5 while  $n > N$  do
6    $H \leftarrow \text{GetHighestSimPair}(M, \hat{R}, W);$ 
7    $\hat{R} \leftarrow \text{MergePaths}(H, \hat{R});$ 
8    $W \leftarrow \text{SetDefaultWeights}(\hat{R}, W);$ 
9    $M \leftarrow \text{CalSimilarityMatrix}(\hat{R}, W);$ 
10   $n \leftarrow \text{Length}(\hat{R});$ 
11 end
12  $D \leftarrow \hat{R};$ 
13 for each path  $d_i \in D$  do
14    $S[i] \leftarrow \text{ExtractCodeSnippet}(d_i, F);$ 
15 end
16 return  $S$ 

```

where e represents the bias. Based on the similarity matrix M , we iteratively merge the two most similar dependency flow paths. After each merge, the dependency flow set $\hat{R}$ and matrix M are updated. This process continues until the number of paths in $\hat{R}$ is no more than N . The result is a set of reconstructed multi-dependency flow fusion paths, denoted as $D = \{d_1, d_2, \dots, d_n\}$. After matching these paths with function F , we obtain the final fusion code snippet S , which contains N multi-dependency flow fusion code fragments. Each fused path is designed to retain the core code semantics of the original dependency flows.

4.3. Model learning

In this section, we use a deep learning model to detect potential vulnerabilities in the fusion code snippet S . To achieve more accurate learning and detection, we designed a multi-level deep learning architecture to effectively handle complex dependency relationships in Fig. 6.

4.3.1. Multi-dependency path representation encoding

We use the transformer-based pre-trained UniXcoder model as the encoder, which is a unified cross-modal code language model that uses a one-to-one mapping method [32]. It converts the AST into a sequence structure that retains all the structural information of the tree.

Given a fusion code snippet $S = \{s_1, \dots, s_n\}$, UniXcoder first transforms the code snippet into a flattened AST token sequence set, denoted as $\tilde{S} = \{\tilde{s}_1, \dots, \tilde{s}_n\}$. Next, UniXcoder extracts the code feature and learns the representation on $\tilde{S}$. UniXcoder uses T transformer layers to tokenize and generate the hidden state H_T . For UniXcoder, three modes are available. We adopt the encoder-only mode with the prefix $p = [\text{Enc}]$. Each transformer layer uses the same transformer unit frame. The transformer layer processes input through multi-head self-attention and feed it into a feedforward neural network [33]. For the l transformer layer, the output of the multi-head self-attention mechanism can be formulated as follows:

$$Q_U = H^{l-1} W^Q, \quad (11)$$

$$K_U = H^{l-1} W^K, \quad (12)$$

$$V_U = H^{l-1} W^V, \quad (13)$$

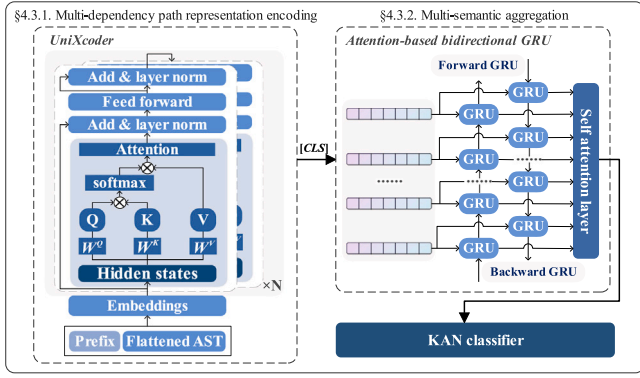


Fig. 6. Model learning.

$$\text{head} = \text{softmax} \left(\frac{Q_U K_U^T}{\sqrt{d_k}} + M \right) V_U, \quad (14)$$

where H^{l-1} is the input from the previous layer. W^Q , W^K , and W^V represent the weight for query, key, and value, respectively. The matrix M is a mask matrix. The mask matrix M is set to 0 in the encoder-only mode, which means that all tokens are allowed to interact. Suppose $\tilde{s}_i = \{\tilde{s}_{i1}, \dots, \tilde{s}_{ik}\}$ is a collection set. This collection belongs to the i th subset sequence in the flattened AST token sequence set $\tilde{S}$. First, it is embedded through the encoder and converted into the input matrix e_{word} . After being processed by T layers of the transformer, it is encoded into a context vector set, denoted as $\tilde{S}^T = \{\tilde{s}_1^T, \dots, \tilde{s}_n^T\}$. Due to the specificity of our downstream tasks, we only need to take the first token of each context-encoded vector $\tilde{s}_i^T$. We obtain the multidependency flow fusion matrix $E \in \mathbb{R}^{n \times d}$ by constructing the first token of each subsequence in the set $\tilde{S}^T$.

4.3.2. Multi-semantic aggregation

To aggregate multiple dependency flow code statements, we use an Attention-BGRU, which learns the dependencies between paths and merges the feature vectors of multiple dependency flow fusion paths. Specifically, given the input multi-dependency flow fusion matrix $E = [e_1, \dots, e_n]^T$, where $e_t \in \mathbb{R}^{d_e}$ is the input vector at the t th time step. The Attention-BGRU computes the forward and backward hidden states at each time step, as follows:

$$\bar{h}_t = \text{GRU}_{\text{forward}}(e_t, \bar{h}_{t-1}), \quad (15)$$

$$\bar{h}_t = \text{GRU}_{\text{backward}}(e_t, \bar{h}_{t+1}), \quad (16)$$

$$h_t = [\bar{h}_t, \bar{h}_t] \in \mathbb{R}^{2d_h}, \quad (17)$$

where h_t is the hidden state at the t th time step, and d_h is the hidden dimension of the GRU.

Next, we calculate the attention weights for each time step t . They quantify the contribution of each hidden state to the final text vector. The attention function is computed through a linear transformation and activation function ($\tanh$), as follows:

$$\alpha_t = \text{softmax}(\tanh(W_{\text{att}} \cdot h_t + b_{\text{att}})), \quad (18)$$

where $W_{\text{att}} \in \mathbb{R}^{2d_h \times 1}$ is the attention weight matrix, and b_{att} is the bias. The attention weight α_t is normalized through softmax to get the importance score for each time step, as follows:

$$\alpha_t = \frac{\exp(\alpha_t)}{\sum_{i=1}^T \exp(\alpha_i)}. \quad (19)$$

Then, we compute the weighted sum of all hidden states h_t and feed them into the dropout layer and the fully connected layer. We get

the context multi-dependency flow fusion vector c , which is used for subsequent classification tasks, as follows:

$$c = W_{fc} \cdot \text{Dropout} \left(\sum_{i=1}^T \alpha_i \cdot h_i \right) + b_{fc}, \quad (20)$$

where W_{fc} represents the weight matrix of the linear layer, and b_{fc} represents the bias. The vector $c \in \mathbb{R}^{d_h}$ captures the global signal of the input sequence. Then we feed it into a series of dropout layers and linear transformations to reduce the possibility of overfitting. Then, we obtain the vector X .

To better capture complex features, we replace the traditional multi-layer perceptron (MLP) with the Kolmogorov–Arnold Network (KAN) as our classifier [25]. The traditional MLP is grounded in the universal approximation theorem. It states that neural networks can approximate any continuous function using combinations of linear transformations and nonlinear activations. In contrast, KAN is inspired by the Kolmogorov–Arnold representation theorem. This theorem indicates that any multivariate function can be represented by addition and univariate functions. KAN no longer uses traditional linear weight matrices, but instead adopts a new approach to represent connection weights. Each connection weight is replaced with a univariate function, typically parameterized by splines. In particular, the univariate functions enable the network to better fit local regions of the input space that require higher resolution. Compared to the traditional MLP, KAN offers better local control in function fitting.

When the final code representation X is obtained, we use the KAN classifier for vulnerability detection, as follows:

$$\Phi_i(x) = W_{\text{base}}^i \cdot \text{BaseActivation}(x) + W_{\text{spline}}^i \cdot \text{SplineBasis}(x), \quad (21)$$

$$\hat{y} = \text{KAN}(x) = (\Phi_{L-1} \circ \Phi_{L-2} \circ \dots \circ \Phi_1 \circ \Phi_0)_x, \quad (22)$$

where $\Phi_i(x)$ represents the combination of linear transformations and piecewise polynomial transformations. We use SiLU as the basic activation function and the basis functions are generated by B-splines. The number of KAN layers L is taken as 64. Finally, we use the typical cross-entropy loss function for vulnerability prediction [34].

5. Experimentals

In this section, we verify the effectiveness of the MDVul through experiments. First, we set up the experiments. Next, we analyze the experimental results based on the two datasets and compare them with other state-of-the-art methods. Finally, we conduct a case study using a real-world vulnerability.

5.1. Experimental setup

5.1.1. Datasets

To ensure a fair comparison, we conduct training and validation using two publicly available datasets: the Ffmpeg+Qemu dataset [35] and the TrVD dataset. In addition, we leverage a subset of the DiverseVul dataset to analyze the model's sensitivity to different vulnerability types. The Ffmpeg+Qemu dataset was created by security experts after over 600 h of manual labeling and cross-validation. Excluding a small amount of data incompatible with Joern, we extracted 26,069 functions for the experiments. The TrVD dataset was built from the Software Assurance Reference Dataset (SARD), a widely used vulnerability database. To evaluate the performance of MDVul across different vulnerability types, Sub_DiverseVul was selected from the DiverseVul dataset [36]. This dataset contains fully labeled, non-overlapping samples with sufficient counts per vulnerability type for sensitivity analysis. Table 2 shows the statistics of the datasets. Ffmpeg+Qemu_p represents the processed version of the Ffmpeg+Qemu dataset, and TrVD refers to the TrVD dataset. Sub_DiverseVul is used to evaluate the performance of the model across different vulnerability types.

Table 2
Statistics of the dataset.

Dataset	Non-vul	Vul	Count	Language
Ffmpeg+Qemu_p	14 117	11 952	26 069	C/C++
TrVD	166 641	98 181	264 822	C/C++
Sub_DiverseVul	131 439	8576	140 015	C/C++

5.1.2. Baseline methods

To validate the superiority of our method, we compared it with the following state-of-the-art methods:

- **VulDeePecker** [15] constructs code slices that are transformed into vectors and fed into a BLSTM for training by extracting library/API function calls from functions [37].
- **SySeVR** [16] proposes four vulnerability syntax features, such as Library/API function call, array usage, pointer usage, and the arithmetic expression. It slices the PDG to generate SeVCs and utilize the BGRU to achieve the best training results.
- **Deign** [14] is a typical graph-based model that integrates AST, CFG, DFG, and NCS into a unified graph. It uses a Gated Graph Recurrent Network (GGRN) to detect code vulnerabilities.
- **LineVul** [12] is a token-based model based on code large language models. It feeds the code sequence into the pre-trained model CodeBERT for detection. LineVul (function level) can be seen as a benchmark model for the pre-trained model CodeBERT.
- **UniXcoder** [32] is a large language model designed for code analysis, and we use it as the benchmark model for Large Code Language Models.
- **EPVD** [18] decomposes the code into several paths by analyzing code execution paths. It encodes these paths using the pre-trained model CodeBERT [30] and captures related features using CNN [38].
- **TrVD** [39] converts the code into an AST and decomposes it into multiple subtrees. It then employs a tree-structured recursive neural network (RvNN) to extract subtree semantics and a Transformer to aggregate and classify.
- **DLAP** [40] uses pretrained models to stimulate adaptive implicit fine-tuning of large language models (LLMs). DLAP combines in-context learning and chain-of-thought techniques to build a dynamic prompt engineering framework. This framework guides LLMs to perform multi-step reasoning and context adaptation in vulnerability detection tasks.

5.1.3. Evaluation metrics

To accurately evaluate the performance of MDVul in vulnerability detection tasks, we use several common evaluation metrics. These metrics quantify the performance of the model in detecting vulnerabilities in code in different datasets. It includes precision, recall, accuracy, and F1 score. The formulas for these metrics are as follows:

$$Precision = \frac{TP}{TP + FP}, \quad (23)$$

$$Recall = \frac{TP}{TP + FN}, \quad (24)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}, \quad (25)$$

$$F1 - Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}, \quad (26)$$

where TP is true positive, FP is false positive, FN is false negative, and TN is true negative.

Table 3
Performance comparisons of different numbers of paths.

Numbers	Accuracy	F1 score	Time costs
3	65.78	66.88	0.22 h/epoch
4	65.54	68.09	0.38 h/epoch
5	66.22	68.23	0.72 h/epoch
6	67.18	67.85	1.22 h/epoch

Table 4
Flows merge under different weighting strategies.

Strategy	Std. deviation	Max-Min range
a (0.01/100)	13.16	34.51
b (all = 1)	13.83	36.10
c (Random)	14.21	36.96

5.1.4. Experiment implementation

We implemented MDVul on Ubuntu 18.04.6 LTS, with an Intel Xeon Silver 4214 CPU @ 2.20 GHz, 64 GB RAM, and an NVIDIA RTX 3090 GPU. The experiments are performed on Python 3.11.5 and PyTorch 2.3.0 [41] with Joern [42] version 4.0.0. To obtain the optimal parameters for MDVul, we trained on the Ffmpeg+Qemu dataset and performed manual tuning. By comparing performance and resource consumption, the most suitable hyperparameter settings were determined.

First, experiments were conducted with different numbers of fusion paths to find the optimal setting. As shown in Table 3, training time increased significantly with more fusion paths. Considering both performance and resource consumption, the number of fusion paths was ultimately set to 5.

Second, to optimize the weighting of flows merge, three different weighting strategies were designed for comparison: (a) assigning 0.01 to the longest path, 100 to the shortest, and 1 to the others during initialization; (b) setting all path weights to 1, ignoring path length and merging solely based on similarity; (c) random initialization with arbitrary weights. To evaluate the effectiveness of each strategy, path balance was used as the main metric, measured by the standard deviation and range of fusion path lengths. Balanced path lengths improve model training stability and generalization by preventing over-reliance on any dominant path. As shown in Table 4, strategy (a) achieved the best results in both metrics, indicating it achieves the most balanced fusion path distribution among the three.

Third, to identify the optimal performance under different hyperparameter settings, we evaluated the impact of several key parameters on model performance. The results are shown in Fig. 7. Fig. 7(a) shows the impact of input length on the performance of UniXcoder. The F1 score increases significantly with longer inputs, reaching the best performance at a length of 1024, indicating that longer sequences provide richer contextual information. Fig. 7(b) presents the effect of the hidden dimension in the Attention-BGRU. The best F1 score is achieved at a dimension of 128, suggesting that an appropriate representation capacity helps capture dependencies within sequences. However, a larger dimension may introduce redundant information and reduce performance. Fig. 7(c) analyzes the impact of different batch sizes. The model performs best when the batch size is set to 8. Fig. 7(d) illustrates the effect of the KAN hidden dimension on model performance. While the overall variation is small, the F1 score is slightly higher at a dimension of 64.

Finally, We set 5 multi-dependency flow fusion paths and batch size to 8. The maximum input token length for UniXcoder is set to 1024. We configure the BGRU hidden dimension to 128 and set the KAN hidden dimension to 64. AdamW is used as an optimizer [43]. We use the F1 score from the validation dataset to select the best model.

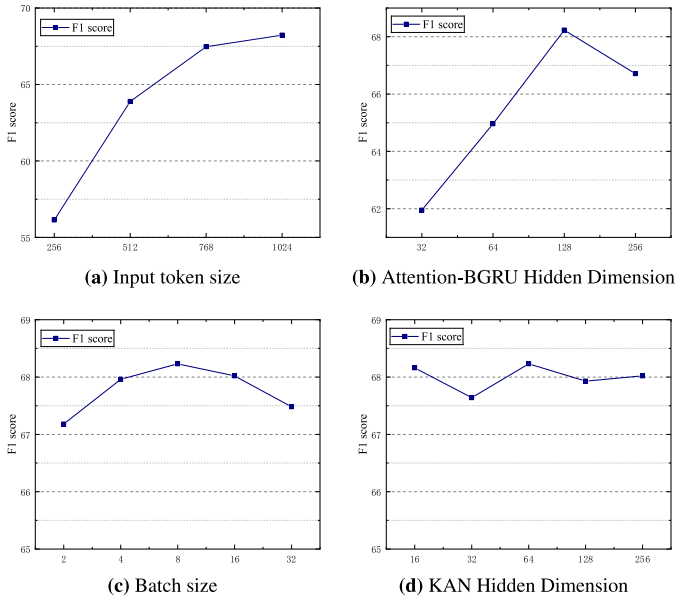


Fig. 7. Impact of different hyperparameter settings on model performance.

5.2. Experimental result

To evaluate our experiments, we propose the following research questions:

RQ 1: How does the overall performance of MDVul compare to other state-of-the-art benchmarks?

RQ 2: How do the different designs of MDVul affect vulnerability detection?

RQ 3: How well does MDVul perform in detecting different types of vulnerabilities?

5.2.1. Model performance analysis (RQ1)

To evaluate the performance of our experiment, we compared MDVul with seven other methods in terms of precision, precision, recall, and F1 score for complex dependency code vulnerability detection. These methods represent the latest benchmarks for different methods in code vulnerability detection. A higher F1 score indicates better overall performance when dealing with unbalanced datasets. The results are shown in Table 5. MDVul surpasses other benchmark models, achieving the highest F1 score. Specifically, it improves over EPVD in precision, accuracy, recall, and F1 score by 7.64%, 9.63%, 5.86%, and 7.96%, respectively, in the Ffmpeg+Qemu dataset.

Based on the experimental results, we will next analyze the reasons behind these results caused by different methods. VulDeePecker and SySeVR perform the worst in these vulnerability detection benchmarks. VulDeePecker performs poorly because it only extracts API call code, missing critical vulnerability information. Although SySeVR contains more potentially vulnerable code information, its performance improvement is not significant. Although SySeVR contains more vulnerability information, the LSTM it uses has significant limitations in its ability to learn such tasks.

LineVul and UniXcoder, as representatives of code large language models, have good generalization and code understanding abilities. They perform well in detecting vulnerabilities in shortcode. When dealing with long code, they will directly truncate code that exceeds the input limit. This way may miss vulnerabilities. Due to these input limitations, their detection performance in complex dependency code is poor. Through comparative experiments, we found that EPVD performs well. The main reason is that it decouples code fragments by analyzing the code execution paths. It uses the pre-trained models

Table 5

Comparison results between MDVul and the baselines.

Datasets	Models	Accuracy	Precision	Recall	F1 score
Ffmpeg+ Qemu_p	MDVul	66.22	61.16	77.15	68.23
	VulDeePecker	40.12	37.45	28.74	32.52
	SySeVR	46.46	42.57	50.36	46.14
	EPVD	61.52	55.79	72.88	63.20
	LineVul	62.46	60.15	60.71	60.43
	UniXcoder	64.85	63.25	60.96	62.09
	Devign	58.42	58.62	65.38	61.82
	TrVD	59.58	56.67	69.34	62.37
	DLAP	63.97	68.69	58.10	62.95
TrVD	MDVul	87.64	91.83	86.73	89.21
	VulDeePecker	86.14	80.47	78.61	79.53
	SySeVR	87.25	82.13	80.14	81.12
	EPVD	87.34	84.51	85.69	85.10
	LineVul	87.97	84.86	81.35	83.07
	UniXcoder	88.26	85.01	82.88	83.93
	Devign	87.61	81.16	79.03	80.08
	TrVD	88.49	90.68	81.97	86.11
	DLAP	88.41	87.69	82.93	85.24

and CNN to aggregate code vulnerability information. This makes it easier for the model to capture code vulnerabilities. Experimental results show that using code pre-trained models enhances the model's understanding and detection of vulnerabilities. However, EPVD still has some shortcomings. Although it alleviates the input limitation issue of CodeBERT, its decoupling of code execution paths leads to excessive code redundancy. This may hinder the pre-trained model's code understanding. Its code representation method only considers the control dependency relationships in the code. As a result, it performs well in certain types of vulnerability and shows poor performance in others. For example, EPVD showed good vulnerability detection ability in CWE-362 [44] and CWE-20 [45], but performed poorly in CWE-476 [46]. Our method is actually inspired by the EPVD's method. We found that the EPVD decoupling path method can fully utilize the pre-trained model's code understanding ability. Although EPVD's method focuses on control dependencies, its structure is less helpful for model learning compared to converting code into a graph.

Devign uses a graph-based model to detect vulnerabilities. Although the graph structure can explicitly capture the structural information of the code, the model's word embedding coding is limited to passing information between lines of code. This limitation might overlook local word relationships in code statements. Although Devign uses compound graph structures to enrich the information in the graph, it inevitably introduces redundancy. Due to the limitations of graph neural networks in handling long-range dependencies, Devign performs poorly when dealing with complex long code fragments.

We observed that TrVD demonstrates strong performance on provided datasets. By converting code into an AST, TrVD captures the program's syntactic structure. Decomposing the AST into subtrees allows for a more fine-grained analysis, which helps capture local patterns and contextual dependencies within different parts of the code. However, both TvNN and Transformer models require significant computational resources.

DLAP achieves performance similar to fine-tuned pretrained models like UniXcoder and LineVul. This shows that combining features from a pretrained model with prompts can help an LLM perform comparably to full fine-tuning. The main advantage of DLAP is that it greatly reduces computational cost by using a lightweight deep learning model and an online LLM for vulnerability detection. However, its performance depends heavily on the quality of the selected models. Inappropriate choices can negatively impact its final performance.

MDVul stands out among these models and performs the best. Compared to traditional methods, MDVul can effectively integrate control flow and data flow information. For complex dependency code, MDVul captures the implicit long-distance dependencies in the code

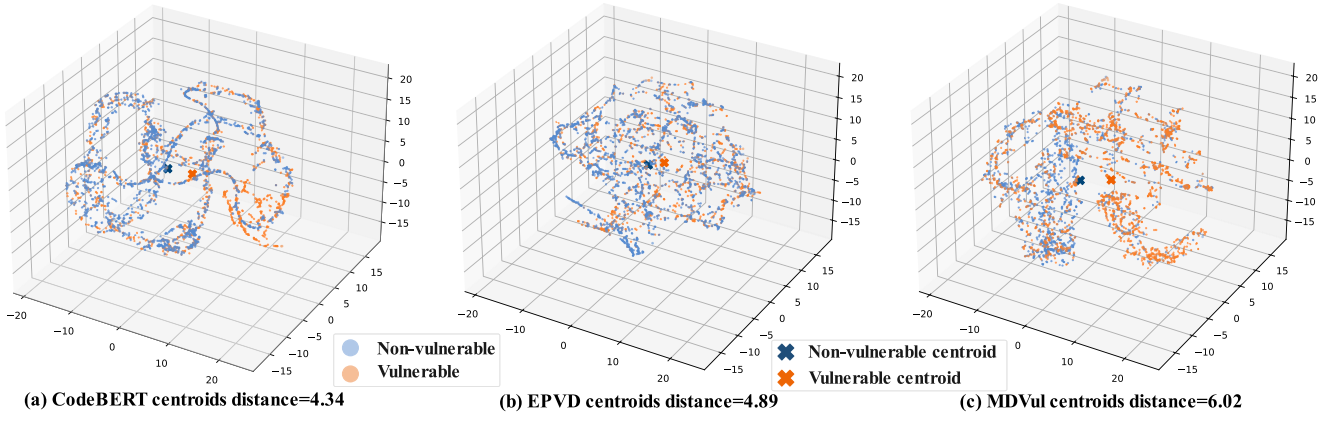


Fig. 8. The t-SNE plot shows the 3D distribution of vulnerable (orange) and non-vulnerable (blue) code representations from various methods. The centroids for each group are marked with 'X', and the centroids' distance reflects the separation between these two groups in the representation space.

through a multi-dependency flow reconstruction strategy. It reduces the information redundancy and preserves the ability to capture key features. MDVul's fusion paths can effectively address information loss and make it more stable in detecting various types of vulnerability. MDVul leverages UniXcoder to improve its understanding of code semantics and vulnerability detection. MDVul combines UniXcoder's code understanding ability with the multi-dependency flow reconstruction. MDVul simplifies complex code relationships by dividing and addressing code fragments separately. As shown in Fig. 8, we use the t-SNE algorithm to visualize the learning ability of code representations for CodeBERT (same as LineVul), EPVD and MDVul [47]. We calculated the Euclidean distance between the centroids of vulnerabilities and non-vulnerabilities for different methods. The data and images show that the MDVul method has the strongest ability to distinguish between vulnerable and non-vulnerable code among these methods.

To evaluate the performance of MDVul's Multi-dependency flow fusion path reconstruction method, We tested the code processing methods of SySeVR, EPVD, and MDVul. The code coverage of the three methods is all over 90%. SySeVR's code slicing method is divided into two stages. First, they extract the relevant code fragments by analyzing the four syntax features to generate SyVCs. Then, they convert them into SeVCs using forward and backward slicing. SeVCs include four types of code segments. SySeVR identifies suspected vulnerability locations without simplifying code logic. EPVD decouples the code by analyzing the execution paths, and each execution path has a simple structure. The paths decomposed by EPVD contain many repetitive parts. EPVD covers vulnerabilities in code with simple control structures but creates highly redundant code representations. MDVul's dependency flow code fragments can be seen as a combination of variable-centered SySeVR and EPVD decoupling paths. MDVul uses the reconstructed dependency flow to replace the original code. MDVul's multi-dependency flow fusion paths merge dependency flows without adding new code fragments. EPVD decoupling paths result in each path containing many redundant parts, while MDVul minimizes redundancy by extracting dependency flows of different variables. Experiments indicate that arrays, pointers, function calls with parameters, and arithmetic expressions involve data flow between variables. Function calls without parameters only have vulnerabilities confined to their internal scope, and they are unrelated to surrounding code.

Our method is centered on variables, and each code fragment contains semantic information based on certain variables. EPVD is also composed of multiple code slices based on control dependencies. SeVCs extract part of a code fragment from the source code based on rules without simplifying dependencies. Fig. 9 compares the performance of three slicing methods in terms of cyclomatic complexity. The line chart shows the average cyclomatic complexity number (Avg-CCN) across intervals for both original and sliced data. MDVul consistently

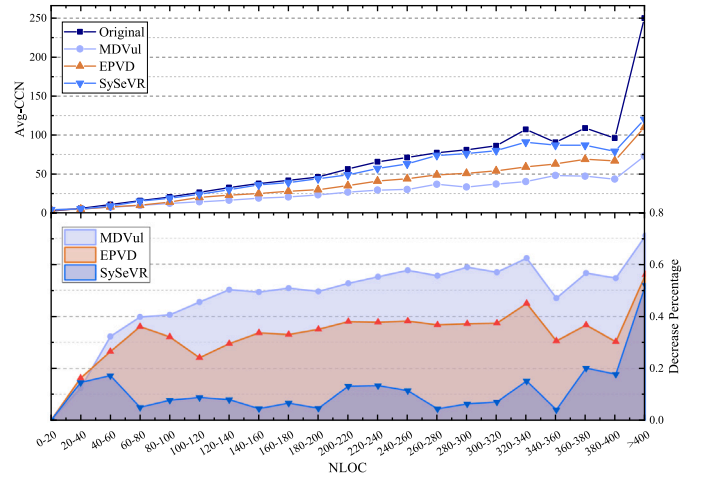


Fig. 9. Comparison of cyclomatic complexity number (CCN) after slicing using three methods. The line chart shows the comparison of data CCN between raw data and sliced data using three methods. The dataset was divided into intervals, with the average cyclomatic complexity number (Avg-CCN) of each interval used as data points. The area part illustrates the percentage decrease in CCN achieved by each method.

achieves the lowest average complexity among the three. This means its generated program slices are simpler and easier to maintain. The area chart shows the percentage reduction in cyclomatic complexity for each method. The results show that MDVul achieves the greatest reduction, demonstrating its effectiveness in simplifying control flow complexity.

Although our method significantly reduces overall cyclomatic complexity, the reduction weakens for large functions with NLOC over 400. This is mainly because a small number of fusion paths makes each path include too much code semantics. It reduces the effectiveness of dependency pruning and limits the simplification of complex control flows. Therefore, increasing the number of fusion paths is an effective strategy for large functions. As shown in Table 6, we compare the complexity reduction for functions with NLOC over 400 under different N settings. The results show that a larger number leads to greater complexity reduction, confirming the effectiveness of this approach for handling large functions.

In summary, The MDVul code slicing method effectively removes code redundancy compared to other slicing methods. The variable-centered multi-dependency flow reconstruction method integrates dependency relationships through fusion paths. It enables the model to better capture key vulnerability information during code representation learning. While SySeVR effectively includes vulnerability information,

Table 6

Results under different numbers of fusion paths (NLOC over 400).

Version	Avg-CCN	Decrease (%)
Original	250.04	–
Slice N = 5	72.81	70.88
Slice N = 6	70.66	71.74
Slice N = 7	69.55	72.18
Slice N = 8	68.16	72.74

Table 7

Performance comparisons of different model architectures.

Models	Accuracy	Precision	Recall	F1 score
MDVul	66.22	61.16	77.15	68.23
MDVul-uFreeze	60.43	51.29	64.57	57.17
MDVul-cFFT	64.14	68.78	62.54	65.51
MDVul-cFreeze	58.09	52.41	60.24	56.05
MDVul-natt	67.14	60.41	78.01	68.09
MDVul-aml	62.78	68.43	66.74	67.57
MDVul-mlp	65.94	61.47	75.28	67.68

it destroys code semantics. This lack of semantics hinders model learning. EPVD cannot fully capture vulnerabilities in complex code due to its unique slicing method and bias towards certain structures.

5.2.2. Ablation experiment analysis (RQ2)

In this subsection, we will discuss how different model architectures of MDVul impact vulnerability detection performance.

In the encoding stage, we use UniXcoder to encode the code representation and update the model parameters using full fine-tuning. In the aggregation stage, we use an Attention-BGRU for information aggregation and vulnerability information extraction. In the final classification stage, we use the KAN network for classification. For each stage, we designed different model architectures: for the encoding stage, we designed three variants. Our main method uses full fine-tuning of UniXcoder as the encoder, so we designed MDVul-uFreeze (with frozen UniXcoder's parameters) and two variants that use CodeBERT as the encoder, namely MDVul-cFFT (full fine-tuning) and MDVul-cFreeze (with frozen CodeBERT parameters). For the aggregation stage, we designed two variants: without the Attention mechanism using BGRU (denoted as MDVul-natt) and using MLP for aggregation (denoted as MDVul-aml). In the classification stage, we designed one variant, using a regular MLP for classification, denoted as MDVul-mlp.

The experimental results on the Ffmpeg+Qemu dataset are shown in Table 7. It is clear that UniXcoder outperforms CodeBERT in code understanding tasks. UniXcoder transforms nodes in the abstract syntax tree (AST) into a sequential format through a one-to-one mapping. This preserves the original semantic and hierarchical structure of the code, allowing the model to better capture syntactic and structural relationships during training. In contrast, CodeBERT has weaker capabilities in structural modeling, which limits its performance on more complex code understanding tasks.

The results show that full fine-tuning outperforms the parameter-freezing strategy. This means that relying solely on the pretrained model's parameters is not enough to handle downstream tasks. Although pretrained models provide a strong feature foundation, further task-specific tuning is needed to fully unlock their potential in vulnerability detection.

In the feature aggregation stage, Attention-BGRU outperforms other aggregation methods. The attention mechanism enhances the model's ability to focus on key parts of the input, improving its perception of important features. Meanwhile, the BGRU extracts sequence information in both forward and backward directions, further strengthening temporal feature representation. As shown by the comparison between MDVul-natt and MDVul-aml, GRU-based feature aggregation performs better than MLP. This further demonstrates the advantage of recurrent neural networks in handling code, which typically contains both sequential and structural information.

In the classification stage, KAN outperforms the traditional MLP (MDVul-mlp). Traditional MLP classifiers build nonlinear mappings using linear transformations followed by fixed activation functions (such as ReLU or SiLU). Their expressive power largely depends on the network's depth and width. KAN moves the activation function from nodes to edges, using learnable univariate functions (such as B-splines) as weights. This gives each layer of the network stronger expressive power structurally, allowing it to model complex feature mappings without relying on excessively deep architectures. In code vulnerability detection, the model needs to distinguish code snippets that are semantically similar but have different security properties. MLP struggles with highly contextual and high-dimensional representations, often hitting a capacity limit or needing extra parameters to fit the data. KAN adapts features using learnable univariate functions on edges. This allows the network to apply different transformations depending on the input, rather than fixed activation patterns. This mechanism helps the model better understand complex logic when used with the features from the encoder and aggregation modules. Experiments show that replacing the MLP classifier with KAN improves performance across multiple metrics. This shows that KAN's flexibility and fine-grained modeling make it more suitable than traditional MLPs for code vulnerability classification.

5.2.3. Vulnerability type sensitivity analysis (RQ3)

The performance of MDVul and other methods in detecting different types of vulnerabilities is shown in Table 8. The results show that MDVul performs well in many types of vulnerabilities. It is especially good at handling complex vulnerabilities, such as Protection Mechanism Failure (CWE-693), Improper Neutralization (CWE-707), and Improper Control of a Resource Through its Lifetime (CWE-664). MDVul is able to capture key data flow and control flow dependencies more effectively when dealing with vulnerability types that rely strongly on contextual information. In CWE-693, MDVul performs well on subclasses such as CWE-352 and CWE-347, achieving detection accuracies of 86.54% and 83.33%, respectively. This type of vulnerability is often caused by the incorrect use or deployment of protection mechanisms, which allows attackers to bypass the system's security logic. Many protection mechanisms are often implemented through logical branches inside functions. Therefore, even though MDVul detects vulnerabilities at the single-function level, it can still learn typical implementation patterns of protection mechanisms within functions. In CWE-707, MDVul also shows strong detection ability for several input validation-related vulnerabilities, achieving detection accuracies of 84.54% and 70.65% in CWE-78 and CWE-20, respectively. MDVul is highly sensitive to cases where external input is not properly handled, enabling it to effectively identify related vulnerabilities. Additionally, in CWE-664, MDVul achieves 81.67% accuracy in CWE-416, which is significantly better than other models. This type of vulnerability usually involves complex pointer operations and the order of resource release. MDVul improves its understanding of complex semantic relationships by combining dependencies between restructured code semantics along different paths. Although MDVul performs well in most types of vulnerabilities, its results in CWE-284 are not the best. For example, CWE-787 means writing data beyond the buffer limit, which can cause crashes or security risks. These vulnerabilities are usually related to pointer errors, array index mistakes, or poor memory management. MDVul performs second best after UniXcoder, possibly because most data samples causing these vulnerabilities have simple logic dependencies. UniXcoder can capture patterned write behaviors and out-of-bound risks well by keeping the sequence structure of the AST, which leads to high detection performance. Fig. 10 illustrates an Internet of Things (IoT) device vulnerability CVE-2024-7582 [48], which lead to buffer overflow (CWE-120) [49]. Lines 13–15 define three pointer variables that accept the access code, data, and accInfo parameters from the post request. Line 16 directly uses strcpy for copying without performing input validation, which poses a risk of buffer overflow. Similarly, src

Table 8

Accuracy comparison between MDVul and baseline models across multiple vulnerability categories.

Pillar	Type	LineVul	UniXcoder	EPVD	MDVul
Protection mechanism failure (CWE-693)	CWE-352	60.47	24.85	64.86	86.54
	CWE-347	–	–	54.99	83.33
	CWE-326	57.74	42.53	66.44	74.38
Improper neutralization (CWE-707)	CWE-79	65.57	30.51	34.75	41.13
	CWE-89	24.77	46.41	56.57	62.67
	CWE-78	7.58	46.21	42.07	84.54
	CWE-20	33.17	25.57	46.10	70.65
Improper control of a resource through its lifetime (CWE-664)	CWE-416	78.23	22.13	53.76	81.67
	CWE-125	31.10	26.65	43.96	53.49
	CWE-22	66.84	27.83	47.58	41.18
	CWE-200	21.99	24.68	54.15	76.92
Improper access control (CWE-284)	CWE-787	29.11	48.01	34.93	36.67
	CWE-269	25.05	54.32	44.63	21.36
	CWE-287	64.06	15.29	66.18	16.32

```

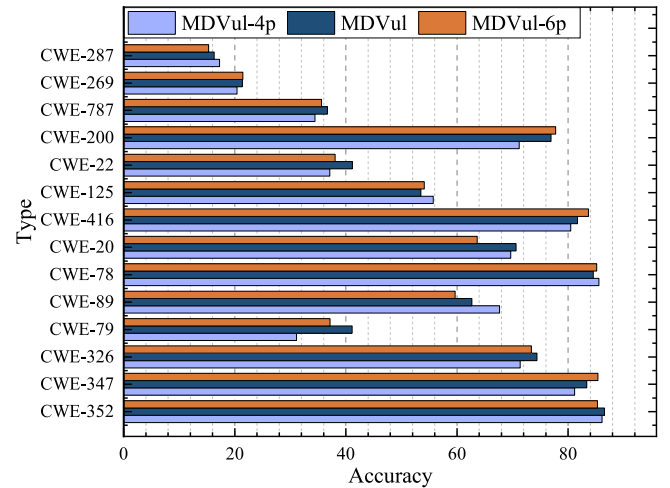
1 int __fastcall formApPortalAccessCodeAuth(int a1)
2 {
3     int v1; //r0
4     char v4[512]; // [sp+2Ch] [bp-5BA0h] BYREF
5     int v6[154]; // [sp+532Ch] [bp-8A0h] BYREF
6     char v9[64]; // [sp+55C4h] [bp-608h] BYREF
7     char v13[1024]; // [sp+5618h] [bp-5B4h] BYREF
8     char *src; // [sp+5BA0h] [bp-24h]
9     const char *v16; // [sp+5BACH] [bp-20h]
10    void *s1; // [sp+5BB0h] [bp-1Ch]
11    ...
12    memset(v6, 0, sizeof(v6));
13    src = (char *)sub_25B04(a1, "accessCode", "pass");
14    s1 = (void *)sub_25B04(a1, "data", &unk_83034);
15    v16 = (const char *)sub_25B04(a1, "acceInfo", &unk_83034);
16    strcpy((char *)&v6[9], src);
17    sub_6B550(a1 + 48, &v6[13]);
18    ...
19    if (*(_BYTE *)s1)
20    {
21        printf("url:%s\n", (const char *)s1);
22        if (!memcmp(s1, "http://", 7u))
23            strcpy(v4, (const char *)s1);
24        else
25            sprintf(v4, "http://%s", (const char *)s1);
26        if (v20 == 26)
27        {
28            printf("url:%s\n", v4);
29            return sub_25EC4(a1, v4);
30        }
31        goto LABEL_21;
32    }
33    if (v20 != 26)
34    {
35        LABEL_21:
36        sprintf(v13, "%s&flag=error&portalCode=%s&authType=%d", v16, v9, 2);
37        return sub_25EC4(a1, v13);
38    }
39    if (HIBYTE(v6[25]))
40        return sub_25EC4(a1, (char *)&v6[25] + 3);
41    else
42        return sub_25EC4(a1, "http://www.ip-com.com.cn");
43 }

```

Fig. 10. CVE-2024-7582.(The yellow shaded statement is the vulnerability).

(lines 19–25) and v16 (line 36) also use string operations such as strcpy and sprintf without length checks.

We also compared the impact of different numbers of fusion paths on performance across vulnerability types. As shown in Fig. 11, different numbers of paths show some differences in certain vulnerability types. For example, in CWE-347, using 6 paths achieves the highest accuracy, slightly better than the default. This shows that increasing the number of merged paths can help capture key semantic information. In CWE-89, MDVul-4p achieves the highest detection accuracy, outperforming both MDVul and MDVul-6p. This shows that for vulnerabilities with more focused structures, using fewer paths reduces redundant noise and improves the model's attention to the merged paths. Overall, the number of merged paths has little impact on model performance. This is mainly because most test samples are moderate in size, and different numbers of paths express variable dependencies similarly. The model can fully capture key contextual information during testing.

**Fig. 11.** Performance comparison between different numbers of fusion paths across multiple vulnerability categories.

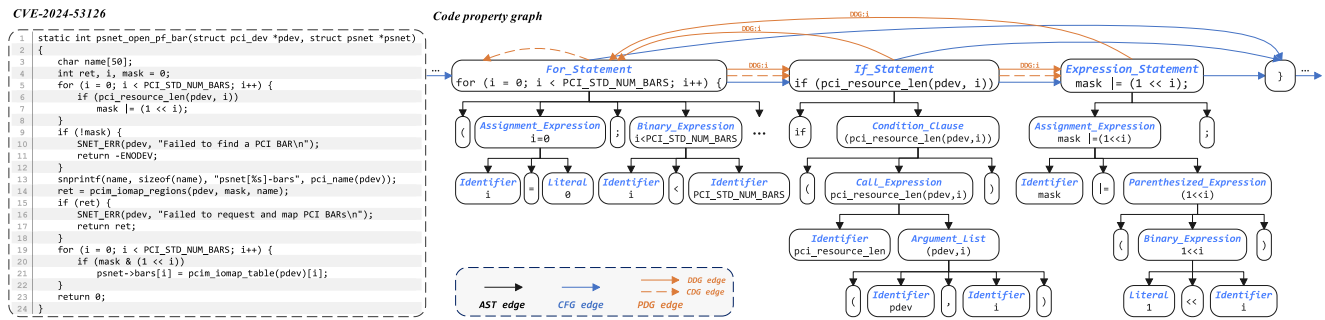
6. Discussion

6.1. Limitations

Model capability. Although the MDVul method performs well in complex dependency code, it still has some limitations when handling extremely complex functions. Specifically, when fusion paths are very long, they may exceed the input length limit of the pretrained model. This can cause some key dependency information to be cut off, affecting the completeness of semantic representation. Therefore, the ability of current method to represent information in extremely complex functions still needs improvement.

Datasets. The datasets we use are different from real-world projects. First, the code in public datasets is usually cleaned or filtered. The code style is more uniform, and the types of vulnerabilities are more focused. This makes it hard to fully show the diversity and complexity of real projects. Second, some data samples are synthetic or automatically labeled, which may cause inaccurate labels or missing context. This affects the model's training and generalization [36].

Vulnerability detection scope. The MDVul method focuses on variable extraction and building multiple dependency flows. It mainly detects vulnerabilities based on variable data and control dependencies. Therefore, its ability to detect vulnerabilities unrelated to variables is limited. In addition, The MDVul captures dependencies only within single functions and does not consider cross-function vulnerabilities.



design a method to construct multi-dependent flow by extracting variables and using flow reconstruction to reduce fusion paths and improve information processing efficiency. We also combine pre-trained models and Attention-BGRU to learn the semantic information of the code and use KAN as a classification model. Our experimental evaluation demonstrated its effectiveness.

CRediT authorship contribution statement

6.2. Future work

Declaration of competing interest

Appendix. Case study

Stage 1: Code representation generation.

1. **Code Property Graph Construction.** We use CVE-2024-53126 as an example. Its CPG is constructed by Joern. The CPG of its partial code (lines 5–7) is shown in Fig. 12.

2. **Dependency Flow Extraction.** We extract the variables from the example and perform dependency flow extraction based on the variables. The set of variables Set_v : [mask, ..., psnet]. The dependency flow paths are shown below:

```
mask = [4, 7, 5, ..., 10, 15, 11]
...
```

$$psnet = [1, 21, 9, \dots, 15, 19, 20]$$

3. Merge Flows For Reconstruction. Following the method above, we use the greedy algorithm-based multi-dependency flow reconstruction method to merge the dependency flows, assuming the number of paths is set to 5. The reconstructed paths are shown below:

are shown below:

[[5, 6, 7, ..., 19, 20, 21], ... [4, 5, 6, ..., 20, 21, 23]]

4. **Flow Sequences Construction.** We match the reconstructed paths with the source functions and add the function header to obtain the final input. This process constructs the multi-dependency flow fusion code fragments. The code fragments are shown below:

```
[“psnet_open_pf_bar(struct...pcim_iomap_table(pdev[i];\n\n}\n}\n}\n}”, ...]
```

Stage 2: Model learning.

5. **Embeddings.** Before embedding, UniXcoder performs tokenization. Converting the code fragments into word vectors with a size of (1, 5, 1024), as shown below:
[[[0, 1179, 1477, ..., 1, 1, 1], [0, 1179, 1477, ..., 1, 1, 1], ...
[0, 1179, 1477, ..., 1, 1, 1]]]
Next, it executes word embedding, positional embedding, type embedding, and finally produces the embedding layer vector with a size of (5, 1024, 768), as shown below:
[[[0.0346, -0.6114, 0.0226, ..., -0.0291, 0.0046, 0.0017], ...
[-0.1575, 0.1073, 0.0388, ..., -0.4149, 0.2303, 0.0682]]]
6. **Encoding Using UniXcoder.** After encoding the vectors through the embedding layer, the vector representation of the multi-dependency flow fusion code fragments is obtained. Its size is (5, 1024, 768), as shown below:
[[[1.4234, -0.5667, -3.3042, ..., 0.3997, 2.1749, 1.7751], ...
[0.8708, -0.6072, -3.6182, ..., -0.0376, 2.4623, 1.2504]]]
7. **Multi-Semantic Aggregation By Attention-BGRU.** We designed an Attention-BGRU to aggregate and learn the feature vectors of multi-dependency flow fusion paths. After applying the activation function Softmax and a series of dropout layers and linear transformations, we obtained the output size of (1, 128). The vector as shown below:
[[-0.4432, 0.0757, -0.6991, ..., 0.3958, 0.4441, -0.3048]]
8. **Classification By KAN.** We use KAN as the output layer of the network, and the output result is 3.4451. Then, we apply the sigmoid function to calculate it, resulting in the final outcome $\hat{y} = 0.97$.
9. **Conclusion.** We need to round the decimal. The result $\hat{y}$ is 1. Therefore, this function is detected as vulnerable code.

Data availability

Data will be made available on request.

References

- [1] OWASP, OWASP top 10 risks for open source software | OWASP foundation. <https://owasp.org/www-project-open-source-software-top-10/>.
- [2] CVE-2024-49854, block, bfc: fix uaf for accessing waker_bfq after splitting. <https://www.cve.org/CVERecord?id=CVE-2024-49854>.
- [3] L. Jiang, G. Misserghy, Z. Su, S. Glondu, DECKARD: Scalable and accurate tree-based detection of code clones, in: 29th International Conference on Software Engineering, Minneapolis, MN, USA, 2007, pp. 96–105, <http://dx.doi.org/10.1109/ICSE.2007.30>.
- [4] Z. Li, D. Zou, S. Xu, H. Jin, H. Qi, J. Hu, VulPecker: an automated vulnerability detection system based on code similarity analysis, in: Proceedings of the 32nd Annual Conference on Computer Security Applications, New York, NY, USA, 2016, pp. 201–213, <http://dx.doi.org/10.1145/2991079.2991102>.
- [5] H. Sun, L. Cui, L. Li, Z. Ding, Z. Hao, J. Cui, P. Liu, VDSimilar: Vulnerability detection based on code similarity of vulnerabilities and patches, Comput. Secur. 110 (2021) 102417, <http://dx.doi.org/10.1016/j.cose.2021.102417>.
- [6] S. Yang, Z. Xu, Y. Xiao, Z. Lang, W. Tang, Y. Liu, Z. Shi, H. Li, L. Sun, Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis, ACM Trans. Softw. Eng. Methodol. 32 (6) (2023) <http://dx.doi.org/10.1145/3604608>.
- [7] S. Han, M. Kim, J. Kang, K. Kim, S. Lee, S. Lee, Similarity-based source code vulnerability detection leveraging transformer architecture: Harnessing cross-attention for hierarchical analysis, IEEE Access 12 (2024) 150295–150307, <http://dx.doi.org/10.1109/ACCESS.2024.3474857>.
- [8] H.-C. Tran, A.-D. Tran, K.-H. Le, DetectVul: A statement-level code vulnerability detection for python, Future Gener. Comput. Syst. 163 (2025) 107504, <http://dx.doi.org/10.1016/j.future.2024.107504>.
- [9] H. Liang, L. Wang, D. Wu, J. Xu, MLISA: A static bugs analysis tool based on LLVM IR, in: 2016 17th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing, Piscataway, NJ, USA, 2016, pp. 407–412, <http://dx.doi.org/10.1109/SNPD.2016.7515932>.
- [10] K. Luckow, R. Kersten, C. Pasareanu, Complexity vulnerability analysis using symbolic execution, Softw. Test. Verif. Reliab. 30 (7–8) (2020) e1716, <http://dx.doi.org/10.1002/stvr.1716>.
- [11] H. Tu, Boosting symbolic execution for heap-based vulnerability detection and exploit generation, in: 2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings, Piscataway, NJ, USA, 2023, pp. 218–220, <http://dx.doi.org/10.1109/ICSE-Companion58688.2023.00059>.
- [12] M. Fu, C. Tantiathamthavorn, LineVul: a transformer-based line-level vulnerability prediction, Assoc. Comput. Mach. (2022) 608–620, <http://dx.doi.org/10.1145/3524842.3528452>.
- [13] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet? IEEE Trans. Softw. Eng. 48 (9) (2022) 3280–3296, <http://dx.doi.org/10.1109/TSE.2021.3087402>.
- [14] Y. Zhou, S. Liu, J. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, 2019, arXiv preprint [arXiv:1909.03496](https://arxiv.org/abs/1909.03496).
- [15] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, 2018, arXiv preprint [arXiv:1801.01681](https://arxiv.org/abs/1801.01681).
- [16] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, IEEE Trans. Dependable Secur. Comput. 19 (4) (2022) 2244–2258, <http://dx.doi.org/10.1109/TDSC.2021.3051525>.
- [17] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, DeepWukong: Statically detecting software vulnerabilities using deep graph neural network, ACM Trans. Softw. Eng. Methodol. 30 (3) (2021) <http://dx.doi.org/10.1145/3436877>.
- [18] J. Zhang, Z. Liu, X. Hu, X. Xia, S. Li, Vulnerability detection by learning from syntax-based execution paths of code, IEEE Trans. Softw. Eng. 49 (8) (2023) 4196–4212, <http://dx.doi.org/10.1109/TSE.2023.3286586>.
- [19] T. Wu, L. Chen, G. Du, D. Meng, G. Shi, UltraVCS: Ultra-fine-grained variable-based code slicing for automated vulnerability detection, IEEE Trans. Inf. Forensics Secur. 19 (2024) 3986–4000, <http://dx.doi.org/10.1109/TIFS.2024.3374219>.
- [20] Common-Weakness-Enumeration, CWE VIEW: Research concepts. <https://cwe.mitre.org/data/definitions/1000.html>.
- [21] O. Ferschke, I. Gurevych, M. Rittberger, FlawFinder: A modular system for predicting quality flaws in wikipedia, in: Conference and Labs of the Evaluation Forum, Rome, Italy, 2012, pp. 1–10, URL: <https://ceur-ws.org/Vol-1178/CLEF2012wn-PAN-FerschkeEt2012.pdf>.
- [22] RATS, RATS is a fast, efficient, and comprehensive econometrics and time series analysis software package. <https://estima.com/ratsmain.shtml>.
- [23] J. Viegas, J. Bloch, Y. Kohno, G. McGraw, ITS4: a static vulnerability scanner for C and C++ code, in: Proceedings 16th Annual Computer Security Applications Conference, Los Alamitos, CA, USA, 2000, pp. 257–267, <http://dx.doi.org/10.1109/ACSAC.2000.898880>.
- [24] D. Chen, Y.-d. Zhang, W. Wei, S.-x. Wang, R.-b. Huang, X.-l. Li, B.-b. Qu, S. Jiang, Efficient vulnerability detection based on an optimized rule-checking static analysis technique, Front. Inf. Technol. Electron. Eng. 18 (3) (2017) 332–345, <http://dx.doi.org/10.1631/FITEE.1500379>.
- [25] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljačić, T.Y. Hou, M. Tegmark, KAN: Kolmogorov–Arnold networks, 2025, arXiv preprint [arXiv:2404.19756](https://arxiv.org/abs/2404.19756).
- [26] C. Li, X. Liu, W. Li, C. Wang, H. Liu, Y. Yuan, U-KAN makes strong backbone for medical image segmentation and generation, 2024, arXiv preprint [arXiv:2406.02918](https://arxiv.org/abs/2406.02918).
- [27] C. Li, W. Li, H. Liu, X. Liu, Q. Xu, Z. Chen, Y. Huang, Y. Yuan, Flaws can be applause: Unleashing potential of segmenting ambiguous objects in SAM, Adv. Neural Inf. Process. Syst. 37 (2024) 45578–45599.
- [28] C. Li, X. Liu, C. Wang, Y. Liu, W. Yu, J. Shao, Y. Yuan, Gtp-4o: Modality-prompted heterogeneous graph learning for omni-modal biomedical representation, in: European Conference on Computer Vision, Springer, 2024, pp. 168–187.
- [29] R. Sennrich, B. Haddow, A. Birch, Neural machine translation of rare words with subword units, 2015, arXiv preprint [arXiv:1508.07909](https://arxiv.org/abs/1508.07909).
- [30] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, et al., Codebert: A pre-trained model for programming and natural languages, 2020, arXiv preprint [arXiv:2002.08155](https://arxiv.org/abs/2002.08155).
- [31] Y. Zhang, B. Wallace, A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification, 2015, arXiv preprint [arXiv:1510.03820](https://arxiv.org/abs/1510.03820).
- [32] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, J. Yin, UniXcoder: Unified cross-modal pre-training for code representation, in: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics, Dublin, Ireland, 2022, pp. 7212–7225, <http://dx.doi.org/10.18653/v1/2022.acl-long.499>.
- [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, L. Kaiser, I. Polosukhin, Attention is all you need, in: Proceedings of the 31st International Conference on Neural Information Processing Systems, NIPS '17, Long Beach, California, USA, 2017, pp. 6000–6010.
- [34] P.-T. De Boer, D.P. Kroese, S. Mannor, R.Y. Rubinstein, A tutorial on the cross-entropy method, Ann. Oper. Res. 134 (2005) 19–67, <http://dx.doi.org/10.1007/s10479-005-5724-z>.
- [35] Ffmpeg+Qemu, Ffmpeg+qemu dataset from devign. <https://sites.google.com/view/devign>.

- [36] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses, RAID '23, Association for Computing Machinery, New York, NY, USA, 2023, pp. 654–668, <http://dx.doi.org/10.1145/3607199.3607242>.
- [37] A. Graves, Long short-term memory, in: Supervised Sequence Labelling with Recurrent Neural Networks, Berlin, Heidelberg, 2012, pp. 37–45, http://dx.doi.org/10.1007/978-3-642-24797-2_4.
- [38] Y. Kim, Convolutional neural networks for sentence classification, 2014, arXiv preprint [arXiv:1408.5882](https://arxiv.org/abs/1408.5882).
- [39] Z. Tian, B. Tian, J. Lv, Y. Chen, L. Chen, Enhancing vulnerability detection via AST decomposition and neural sub-tree encoding, Expert Syst. Appl. 238 (2024) 121865, <http://dx.doi.org/10.1016/j.eswa.2023.121865>.
- [40] Y. Yang, X. Zhou, R. Mao, J. Xu, L. Yang, Y. Zhang, H. Shen, H. Zhang, DLAP: A deep learning augmented large language model prompting framework for software vulnerability detection, J. Syst. Softw. 219 (2025) 112234, <http://dx.doi.org/10.1016/j.jss.2024.112234>, URL: <https://www.sciencedirect.com/science/article/pii/S0164121224002784>.
- [41] PyTorch, An end-to-end machine learning framework. <https://pytorch.org/>.
- [42] Joern, The bug hunter's workbench. <https://joern.io/>.
- [43] I. Loshchilov, F. Hutter, Decoupled weight decay regularization, 2019, [arXiv:1711.05101](https://arxiv.org/abs/1711.05101). URL: <https://arxiv.org/abs/1711.05101>.
- [44] CWE-362, Concurrent execution using shared resource with improper synchronization. <https://cwe.mitre.org/data/definitions/362.html>.
- [45] CWE-20, Improper input validation. <https://cwe.mitre.org/data/definitions/20.html>.
- [46] CWE-476, NULL pointer dereference. <https://cwe.mitre.org/data/definitions/476.html>.
- [47] L. Van der Maaten, G. Hinton, Visualizing data using t-SNE, J. Mach. Learn. Res. 9 (86) (2008) 2579–2605, URL: <http://jmlr.org/papers/v9/vandermaaten08a.html>.
- [48] CVE-2024-7582, Tenda i22 apPortalAccessCodeAuth formApPortalAccessCodeAuth buffer overflow. <https://www.cve.org/CVERecord?id=CVE-2024-7582>.
- [49] CWE-120, Buffer copy without checking size of input. <https://cwe.mitre.org/data/definitions/120.html>.
- [50] I. Beltagy, M.E. Peters, A. Cohan, Longformer: The long-document transformer, 2020, [arXiv:2004.05150](https://arxiv.org/abs/2004.05150). URL: <https://arxiv.org/abs/2004.05150>.
- [51] M. Zaheer, G. Guruganesh, A. Dubey, J. Ainslie, C. Alberti, S. Ontanon, P. Pham, A. Ravula, Q. Wang, L. Yang, A. Ahmed, Big bird: Transformers for longer sequences, 2021, [arXiv:2007.14062](https://arxiv.org/abs/2007.14062). URL: <https://arxiv.org/abs/2007.14062>.